

EXPT NO: 3	IMPLEMENTATION OF CONVOLUTIONAL NEURAL NETWORKS (CNNs) FOR IMAGE CLASSIFICATION
DATE: 5-8-2026	

PRE-LAB QUESTIONS (PROVIDE BRIEF ANSWERS TO THE FOLLOWING QUESTIONS)

1. What is a Convolutional Neural Network (CNN)?

A Convolutional Neural Network (CNN) is a specialized class of deep learning artificial neural networks primarily designed for processing structured grid data, such as digital images. It operates by employing mathematical convolution operations in place of general matrix multiplication in at least one of its network layers. This mathematical framework enables the network to automatically and adaptively learn spatial hierarchies of features from raw input data. Consequently, CNNs maintain the structural integrity of an image by processing localized regions, making them highly effective for rigorous computer vision applications.

2. What are the functions of convolution and pooling layers in a CNN?

The convolution layer serves as the core feature extraction mechanism of a CNN, operating by sliding learnable mathematical filters across the input tensor. By computing the dot product between the filter weights and the localized input pixels, it produces feature maps that capture fundamental spatial representations.

- **Feature Extraction:** Extracts low-level properties such as edges, spatial gradients, and textures in the initial network layers.
- **Hierarchical Learning:** Constructs high-level conceptual features in deeper layers by aggregating the low-level spatial data.
- **Parameter Efficiency:** Reduces the total number of free parameters significantly through continuous weight sharing across the spatial dimensions.

The pooling layer, typically applied subsequent to a convolution operation, performs non-linear downsampling to systematically reduce the spatial dimensions (width and height) of the input volume. This process summarizes the presence of specific features within distinct patches of the feature map, preserving essential structural information while discarding exact coordinate data.

- **Dimensionality Reduction:** Decreases computational load and memory allocation by systematically reducing the network's spatial resolution.
- **Overfitting Mitigation:** Minimizes the risk of overfitting by providing an abstracted, generalized form of the learned spatial representation.
- **Translation Invariance:** Ensures the algorithm mathematically recognizes key features regardless of minor positional shifts within the frame.

3. Differentiate between image classification and object detection.

Feature	Image Classification	Object Detection
Primary Objective	Assigns a single categorical label to the entire image.	Identifies and locates multiple specific objects within a single image.
Output Format	A discrete class label and a corresponding probability score.	Bounding box coordinates ($x, y, width, height$) alongside class labels and confidence scores.
Spatial Localization	Does not provide information regarding where the object is located in the frame.	Explicitly localizes the exact coordinates of the detected objects within the image grid.
Algorithmic Complexity	Less computationally intensive; standard CNN architectures (e.g., ResNet, VGG) are sufficient.	Highly computationally intensive; requires specialized architectures (e.g., YOLO, Faster R-CNN) to process bounding box regression.
Instance Handling	Processes the image as a single global instance.	Processes and distinguishes between multiple instances of the same or different classes simultaneously.

4. Why is image preprocessing important before training a CNN model?

Image preprocessing is a critical preliminary stage in the computer vision pipeline that transforms raw image data into a standardized mathematical format optimized for neural network ingestion. Because raw images frequently exhibit severe variations in scale, illumination, and dimensional ratios, applying systematic transformations is mathematically necessary. This standardization ensures the underlying algorithm focuses its computational resources on extracting relevant spatial features rather than attempting to adapt to arbitrary data inconsistencies.

- **Normalization and Scaling:** Centers data and scales pixel intensity values to a standard continuous range (e.g., 0 to 1), accelerating gradient descent convergence and preventing numerical instability.
- **Dimensionality Standardization:** Resizes all input matrices to the exact fixed tensor dimensions required by the specific CNN's input layer architecture.
- **Noise Reduction:** Applies specialized filtering algorithms to eliminate high-frequency artifacts and variance that could disrupt accurate feature extraction.
- **Data Augmentation:** Generates synthetic geometric variations (such as rotations and translations) to expand the training distribution, thereby improving model generalization.

5. What are the advantages of using benchmark datasets such as CIFAR-10 or Kaggle image datasets.

Benchmark datasets, such as CIFAR-10 or curated repositories on platforms like Kaggle, provide standardized, rigorously annotated data structures essential for algorithmic training and validation. Utilizing these established resources accelerates the engineering lifecycle by eliminating the highly resource-intensive phases of primary data acquisition, cleaning, and manual semantic labeling. Consequently, this enables developers to dedicate their focus directly toward architectural optimization, hyperparameter tuning, and model deployment.

- **Standardized Evaluation:** Establishes rigorous baseline metrics, enabling direct and objective performance comparisons across different network architectures and research methodologies.
- **High-Quality Annotations:** Provides accurate, verified ground-truth labels which are strictly required for effective supervised learning operations and loss calculation.
- **Pre-trained Model Compatibility:** Facilitates efficient transfer learning pipelines, as the majority of foundational network weights are initialized using these standard data distributions.
- **Community Validation:** Offers extensive peer-reviewed documentation, validated implementation strategies, and robust statistical benchmarks for precise troubleshooting and algorithmic refinement.

AIM

To design, implement, and evaluate a Convolutional Neural Network (CNN) for image classification using a benchmark image dataset.

SOFTWARE REQUIREMENTS

- Python
- TensorFlow/Keras or PyTorch
- Google Colab
- Matplotlib
- Kaggle Dataset (e.g., CIFAR-10, Cats vs Dogs, Intel Image Classification)

OBJECTIVES

Students will be able to:

- Understand the architecture of Convolutional Neural Networks.
- Implement a CNN model for image classification.
- Train and evaluate a CNN using a real-world image dataset.
- Visualize training and validation performance.
- Analyze the effectiveness of CNNs for image recognition tasks.

PROCEDURE

PART A: DATASET PREPARATION

- Perform the following tasks:
- Download an image dataset from Kaggle or use CIFAR-10.
- Load the dataset.
- Preprocess images by resizing and normalization.
- Split the dataset into training, validation, and testing sets.

IMPLEMENTATION OF CONVOLUTIONAL NEURAL NETWORKS (CNNs) FOR IMAGE CLASSIFICATION

by Rohith John M | 24BAD100

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
```

```
# Downloading the dataset (CIFAR-10 is built into Keras)
(train_images_full, train_labels_full), (test_images, test_labels) = datasets.cifar10.load_data()
print("Download complete")
# Preprocessing: Normalize pixel values to be between 0 and 1
train_images_full = train_images_full / 255.0
test_images = test_images / 255.0

# Split dataset into training and validation sets (using 5000 images for validation)
val_images = train_images_full[:5000]
val_labels = train_labels_full[:5000]
train_images = train_images_full[5000:]
train_labels = train_labels_full[5000:]
```

```
... Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ————— 3627s 21us/step
Download complete
```

PART B: CNN MODEL IMPLEMENTATION

Build a CNN model consisting of:

- Convolution layers
- ReLU activation
- Max Pooling layers
- Fully Connected layers
- Softmax output layer

Compile the model using:

- Appropriate optimizer
- Cross-entropy loss function

```

# Building the CNN architecture
model = models.Sequential([
    # Convolutional layers with ReLU activation and Max Pooling
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation='relu'),

    # Fully Connected layers
    layers.Flatten(),
    layers.Dense(64, activation='relu'),

    # Softmax output layer for 10 classes
    layers.Dense(10, activation='softmax')
])

# Compiling the model with optimizer and cross-entropy loss
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

```

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"

```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_1 (Conv2D)	(None, 13, 13, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 64)	0
conv2d_2 (Conv2D)	(None, 4, 4, 64)	36,928
flatten (Flatten)	(None, 1024)	0
dense (Dense)	(None, 64)	65,600
dense_1 (Dense)	(None, 10)	650

```

Total params: 122,570 (478.79 KB)
Trainable params: 122,570 (478.79 KB)
Non-trainable params: 0 (0.00 B)

```

PART C: MODEL TRAINING AND EVALUATION

Train the CNN model.

Record:

- Training accuracy
- Validation accuracy
- Training loss
- Validation loss
- Testing accuracy

```

print("\nInitiating Model Training...")
# Train the model and record history
history = model.fit(train_images, train_labels, epochs=10,
                    validation_data=(val_images, val_labels))

# Evaluate on testing set
test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=2)
print(f'\nFinal Testing Accuracy: {test_acc:.4f}')

# Plotting Accuracy and Loss vs Epoch
plt.figure(figsize=(14, 5))

# Accuracy Plot
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Training Accuracy', marker='o')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy', marker='o')
plt.title('Accuracy vs Epoch')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.grid(True)
plt.legend(loc='lower right')

# Loss Plot
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Training Loss', marker='o')
plt.plot(history.history['val_loss'], label='Validation Loss', marker='o')
plt.title('Loss vs Epoch')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.grid(True)
plt.legend(loc='upper right')

plt.tight_layout()
plt.show()

```

```

Initiating Model Training...
Epoch 1/10
1407/1407 ————— 63s 43ms/step - accuracy: 0.4478 - loss: 1.5096 - val_accuracy: 0.5762 - val_loss: 1.2050
Epoch 2/10
1407/1407 ————— 80s 42ms/step - accuracy: 0.5934 - loss: 1.1474 - val_accuracy: 0.6080 - val_loss: 1.1167
Epoch 3/10
1407/1407 ————— 82s 42ms/step - accuracy: 0.6511 - loss: 0.9950 - val_accuracy: 0.6604 - val_loss: 0.9682
Epoch 4/10
1407/1407 ————— 58s 41ms/step - accuracy: 0.6866 - loss: 0.8963 - val_accuracy: 0.6752 - val_loss: 0.9227
Epoch 5/10
1407/1407 ————— 81s 41ms/step - accuracy: 0.7112 - loss: 0.8264 - val_accuracy: 0.6770 - val_loss: 0.9552
Epoch 6/10
1407/1407 ————— 57s 41ms/step - accuracy: 0.7321 - loss: 0.7659 - val_accuracy: 0.6994 - val_loss: 0.8759

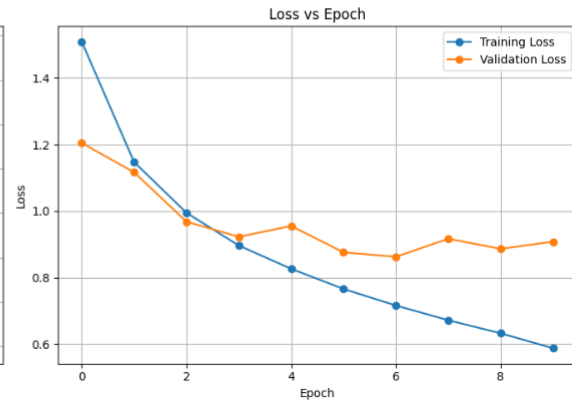
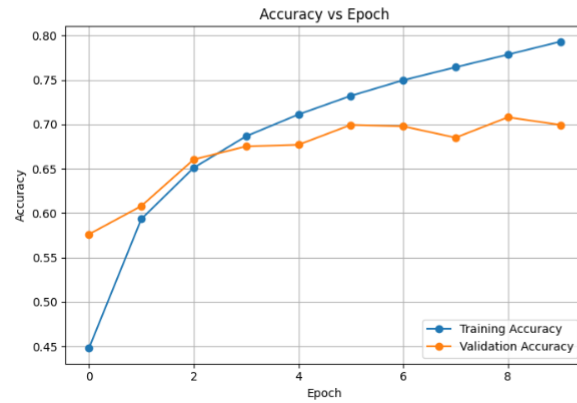
```

Plot:

- Accuracy vs Epoch
- Loss vs Epoch

Epoch 6/10
 1407/1407 — 57s 41ms/step - accuracy: 0.7321 - loss: 0.7659 - val_accuracy: 0.6994 - val_loss: 0.8759
 Epoch 7/10
 1407/1407 — 57s 40ms/step - accuracy: 0.7497 - loss: 0.7161 - val_accuracy: 0.6978 - val_loss: 0.8626
 Epoch 8/10
 1407/1407 — 83s 41ms/step - accuracy: 0.7644 - loss: 0.6718 - val_accuracy: 0.6850 - val_loss: 0.9168
 Epoch 9/10
 1407/1407 — 62s 44ms/step - accuracy: 0.7786 - loss: 0.6326 - val_accuracy: 0.7080 - val_loss: 0.8866
 Epoch 10/10
 1407/1407 — 78s 42ms/step - accuracy: 0.7932 - loss: 0.5876 - val_accuracy: 0.6994 - val_loss: 0.9081
 313/313 - 3s - 11ms/step - accuracy: 0.7036 - loss: 0.9126

Final Testing Accuracy: 0.7036



PART D: PERFORMANCE ANALYSIS

Analyze:

- Classification accuracy
- Confusion Matrix
- Sample predictions
- Misclassified images

Discuss the strengths and limitations of CNNs for image classification.

```

print("\nGenerating Predictions for Analysis...")
predictions = model.predict(test_images)
predicted_classes = np.argmax(predictions, axis=1)

# Define the CIFAR-10 class names explicitly
class_names = ['Airplane', 'Automobile', 'Bird', 'Cat', 'Deer',
               'Dog', 'Frog', 'Horse', 'Ship', 'Truck']

# Generate and plot Confusion Matrix
cm = confusion_matrix(test_labels, predicted_classes)
plt.figure(figsize=(10,8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=class_names, yticklabels=class_names)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix on Test Data')
plt.show()

print("\nClassification Report (Precision, Recall, F1-Score):")
print(classification_report(test_labels, predicted_classes, target_names=class_names))

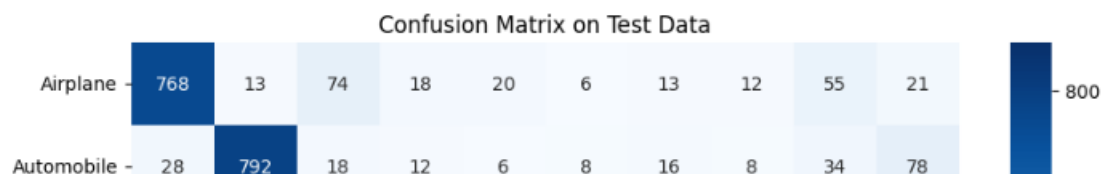
# Visualize Sample Predictions (including misclassified ones)
plt.figure(figsize=(15, 6))
for i in range(10):
    plt.subplot(2, 5, i+1)
    plt.imshow(test_images[i])
    true_label = class_names[test_labels[i][0]]
    pred_label = class_names[predicted_classes[i]]

    # Highlight misclassifications in red
    color = 'blue' if true_label == pred_label else 'red'

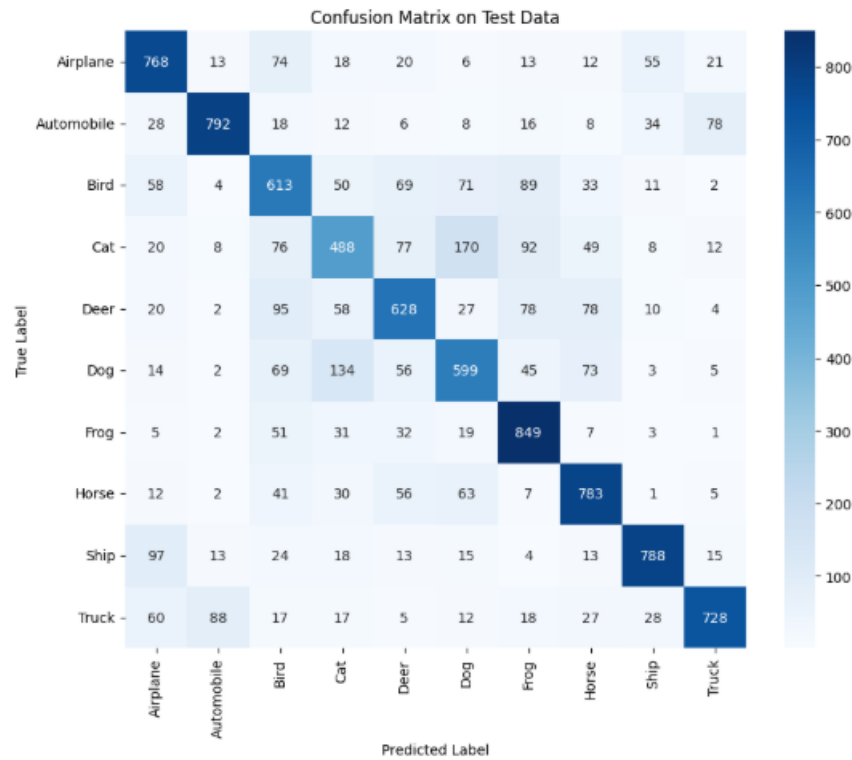
    plt.title(f"True: {true_label}\nPred: {pred_label}", color=color)
plt.axis('off')
plt.tight_layout()
plt.show()

```

Generating Predictions for Analysis...
 313/313  10s 31ms/step



Generating Predictions for Analysis...
313/313 10s 31ms/step



Classification Report (Precision, Recall, F1-Score):

	precision	recall	f1-score	support
Airplane	0.71	0.77	0.74	1000
Automobile	0.86	0.79	0.82	1000
Bird	0.57	0.61	0.59	1000
Cat	0.57	0.49	0.53	1000
Deer	0.65	0.63	0.64	1000

Classification Report (Precision, Recall, F1-Score):

	precision	recall	f1-score	support
Airplane	0.71	0.77	0.74	1000
Automobile	0.86	0.79	0.82	1000
Bird	0.57	0.61	0.59	1000
Cat	0.57	0.49	0.53	1000
Deer	0.65	0.63	0.64	1000
Dog	0.61	0.66	0.68	1000
Frog	0.70	0.85	0.77	1000
Horse	0.72	0.78	0.75	1000
Ship	0.84	0.79	0.81	1000
Truck	0.84	0.73	0.78	1000
accuracy	0.71	0.70	0.70	10000
macro avg	0.71	0.70	0.70	10000
weighted avg	0.71	0.70	0.70	10000



Collab link:

<https://github.com/RohithJohnM/24ADI006-Deep-Learning/blob/main/Exp%203/Implementing CNN.ipynb>

EXPECTED OUTCOME

Students will gain practical experience in designing, training, and evaluating CNN models for image classification and understand how CNN architecture influences classification performance.

POST-LAB QUESTIONS (PROVIDE BRIEF ANSWERS TO THE FOLLOWING QUESTIONS)

1. Why are convolution layers more effective than fully connected layers for image processing?

Convolutional layers process visual data by preserving the inherent spatial grid structure of the input, whereas fully connected dense layers mandate flattening the multidimensional array into a single one-dimensional vector. Flattening immediately destroys the structural relationships between neighboring pixels, rendering dense networks highly inefficient at recognizing localized geometric shapes. Furthermore, fully connected networks require a separate weight for every single pixel interaction, resulting in a mathematical parameter explosion that rapidly exhausts memory and causes severe overfitting on high-resolution matrices.

- **Parameter Sharing:** Convolutional filters apply the exact same mathematical weights across all spatial locations of the input matrix, drastically reducing the total parameter count.
- **Local Receptive Fields:** The network architecture restricts hidden units to only process inputs from a small, localized sub-region, forcing the extraction of foundational features like edges and corners.
- **Spatial Hierarchy:** Sequential stacking of convolutions mathematically aggregates low-level localized features into high-level global representations in the deeper network layers.

2. How does Max Pooling improve CNN performance?

Max pooling is a spatial downsampling mathematical operation that slides a fixed-size window (typically 2×2) over the feature map and extracts only the maximum numerical activation value within that specific region. This operation systematically discards less significant data, ensuring that the subsequent layers receive only the most prominent structural features extracted by the preceding convolution filter. By performing this reduction, the network fundamentally lowers its memory footprint and execution time while creating a more robust and generalized mathematical representation of the input.

- **Dimensionality Reduction:** Mathematically halves the spatial width and height of the tensor, significantly reducing the downstream computational matrix multiplications.
- **Translation Invariance:** Ensures the network detects the targeted feature accurately even if the object is slightly shifted or misaligned within the input frame.
- **Overfitting Mitigation:** Constrains the capacity of the model to memorize exact pixel locations and noise, forcing the algorithm to learn abstract, generalized representations.

3. Which hyperparameters significantly affected your CNN model performance?

The optimization trajectory and final validation accuracy of the network are heavily dictated by the mathematical configuration of specific algorithmic constraints defined prior to compilation. Adjusting these parameters directly manipulates the gradient descent convergence rate and the feature extraction capacity of the convolutional filters. For the defined sequential architecture, specific bounding values governed how effectively the loss function minimized the categorical crossentropy error over the execution cycle.

- **Epoch Count:** Defining exactly 10 training cycles established a strict boundary for iterative learning, balancing the convergence rate against the available computational execution time.
- **Filter Dimensions and Counts:** Configuring the depth from 32 to 64 feature maps in successive layers determined the specific volume of spatial patterns the network could simultaneously process.
- **Optimizer Learning Rate:** The internal step-size utilized by the Adam optimization algorithm dictated the exact magnitude of weight updates during mathematical backpropagation.

4. What challenges did you encounter while training the CNN model?

Executing a deep learning pipeline requires managing environmental variables, strict mathematical array formatting, and cloud hardware execution constraints. Interruptions or out-of-order cell execution in a segmented notebook environment frequently cause strict namespace errors, where visualization functions fail due to undefined categorical arrays. Additionally, extracting large-scale image repositories requires optimized data ingestion, as massive archive files are prone to severe server-side bandwidth throttling if not routed through dedicated infrastructure mirrors.

- **Variable Scoping Exceptions:** Halting execution prior to mapping discrete string labels to their mathematical indices causes critical `NameError` faults during visualization and matrix rendering.
- **Data Ingestion Bottlenecks:** Direct extraction of high-volume zip datasets heavily throttles processing environments, necessitating the use of high-speed educational cloud mirrors to bypass download timeouts.
- **Class Confusion:** Distinguishing morphologically similar labels mathematically requires complex feature extraction that strains basic sequential architectures, resulting in occasional misclassifications.

5. Suggest two improvements that can increase the classification accuracy of your CNN.

Elevating the baseline 70.36% classification accuracy requires modifying both the data ingestion pipeline and the mathematical depth of the network architecture. Expanding the dataset forces the algorithm to learn highly generalized features, while structural adjustments prevent gradient degradation during prolonged backpropagation. Integrating these methodologies systematically mitigates validation loss plateauing.

- **Data Augmentation Integration:** Applying randomized geometric transformations, including horizontal flipping, zooming, and minor rotational shifts via `ImageDataGenerator`, synthetically scales the training distribution to prevent overfitting.
- **Architectural Deepening with Regularization:** Implementing advanced techniques such as Batch Normalization and Dropout layers allows for stacking additional convolutional blocks without introducing severe mathematical instability.

ASSESSMENT

Description	Max Marks	Marks Awarded
Pre Lab Exercise	5	
In Lab Exercise	10	
Post Lab Exercise	5	
Viva	10	

Total	30	
Faculty Signature		